Framework-Decoupled MoE Load Balancing for High-Performance LLM Inference

Problem statement:

MoE inference performance is limited by expert-parallel load imbalance. Static expert placement helps, but live batches can still overload a subset of EP ranks, shared-expert work can remain poorly distributed, and replicated routed experts need per-batch routing decisions. At the same time, each new load-balancing idea is usually integrated separately into SGLang, vLLM, TensorRT-LLM, or other runtimes, creating duplicated engineering work and slowing product adoption.

Importance:

Large MoE models are becoming central to high-throughput LLM serving. A portable MoE load balancer can improve NVIDIA GPU utilization, reduce serving cost, and make new MoE algorithms easier to productize across inference frameworks. The work also creates a common path from research prototypes to production stacks such as SGLang, vLLM, and TensorRT-LLM, reducing the N algorithms x M frameworks integration cost.

Prior work:

The design builds on DeepSeek Expert Parallelism Load Balancer (EPLB)/Linear-Programming-based Load Balancer (LPLB), SGLang DeepEP and Waterfill work, vLLM EPLB support, TensorRT-LLM Wide-EP/online EPLB, and recent research including topology-aware EPLB, CRAFT [1], MoETuner [2], Metro [3], Libra [4], and PROBE [5]. These systems demonstrate strong individual techniques, but their implementations are tied to specific runtime APIs. This submission focuses on turning those ideas into a framework-neutral load-balancing layer with thin runtime adapters.

Technical solution:

We built a framework-decoupled MoE load balancer with three orthogonal layers. L1 performs static expert placement and replica layout planning. L2 performs per-batch token routing while preserving the router top-k and only changing physical execution choices. L3 reserves a future surface for prediction-driven expert prefetch or runtime migration. The core API uses framework-neutral contracts such as PlacementProvider.snapshot, PlacementPlanner.plan, and TokenRouter.route_tokens, while adapters translate to SGLang, vLLM, or TensorRT-LLM-specific data structures. The SGLang POC integrates L1 (EPLB) and L2 (Waterfill and LPLB) and validates the adapter boundary end to end.

Quantitative benefits:

On two-node H20 DeepSeek-V3/R1-style serving workloads, the latest SGLang/DeepEP measurements show Waterfill improving total throughput by +1.48% to +4.66% across MMLU, GPQA, and GSM8K. LPLB improves redundant-EPLB configurations by up to +7.34%, with the strongest result on GSM8K red32. On DeepSeek V4 Flash, Waterfill improves the best measured point from 49,253 tok/s to 51,677 tok/s (+4.92%). These gains preserve model semantics because the router top-k is unchanged; only the physical shared-expert rank or valid routed-expert replica is selected differently.